

Automatic Certification of the Active Corner Method for Collision Avoidance

Nishant Kheterpal, J. Tanner Slagel, Elanor Tang, Serra Dane, and
Jean-Baptiste Jeannin

¹ University of Michigan, Ann Arbor, Michigan, USA
{`nskh`, `sdane`, `jeannin`}@umich.edu

² NASA Langley Research Center, Hampton, Virginia, USA
`joseph.t.slagel@nasa.gov`

³ Carnegie Mellon University, Pittsburgh, Pennsylvania, USA
`elanor@cmu.edu`

Abstract. Formal methods are particularly useful to guarantee safety before deployment in safety-critical tasks. Ensuring absence of collision for vehicles such as airplanes, cars, or robots is one common application of formal verification. However, many methods for verifying collision avoidance model a vehicle as a moving point mass, though the real vehicles have non-zero area. Kheterpal et al. proposed a novel algorithm (the *active corner method*) for verifying collision avoidance for polygonal objects moving in the plane, along with a Python implementation. That implementation was not verified or certified, relying instead on a pen-and-paper proof of correctness. In this work, we describe an extension of the active corner method to automatically generate machine-checkable proof certificates of correctness for specific instances within the Prototype Verification System (PVS). This work briefly discusses the original active corner method proof approach, presents a novel algebraic approach that generalizes to any convex polygon, details the certification process with examples in PVS, and provides a case study using the differential dynamic logic integration within PVS. Our implementation, certificate examples, and proof scripts are available in the supplementary material.

1 Introduction

In safety-critical hybrid systems such as medical devices, power systems, automated vehicles, and airplanes, formal verification helps to assure correct, safe operation and to prevent harm or loss of life. This paper concerns collision avoidance verification, a key task in hybrid systems such as navigation for autonomous robotics in both aerospace and ground settings. In particular, we consider verification for collision avoidance when the object in question has non-zero area. Many approaches for collision avoidance or hybrid system safety reason about point objects, which is unrealistic for robotics applications where hardware systems (e.g., drones, wheeled robots, and grasping arms) have volume and mass [32,5,23].

Prior work [28] introduced the *active corner method* for verifying collision avoidance for polygons moving along known planar trajectories. That work also provided a geometric pen-and-paper proof of correctness and completeness, as well as an open-source implementation in Python. The active corner method produces a quantifier-free geometric representation of the *safe region*: the set of locations where a point obstacle will not collide with a (convex) polygonal object translating along a planar trajectory. However, there is currently a gap between the paper proof and the implementation; the Python code could contain bugs or incorrectly implement the method. Thus, whether errors arise due to existing bugs in libraries used by the implementation, programming errors, or mistakes in the specification, we need further guarantees for deployment in safety-critical settings.

Formal methods provide a way to bridge the *trust gap* between the Python implementation and the pen-and-paper proof. Approaches include full-scale reimplementation of algorithms in formally-verified languages or platforms, such as Rocq, the Prototype Verification System (PVS) [42], Isabelle [40], and others; or formally certifying the correctness of a tool or its outputs. This work presents a **certification** of the active corner method for collision avoidance in PVS, a tool which couples a formal specification language with an interactive theorem prover [42,43]. PVS has long been applied to the formal verification of safety-critical systems, particularly in the domain of air traffic management and unmanned aircraft systems (UAS) [15][16][34]. Overall, the body of work on PVS in the context of UAS detect-and-avoid, collision avoidance, and certification demonstrates its suitability for real-world, safety-critical aerospace applications [36]. PVS proof certificates have been the focus of past research; for example, they can be generated when statically certifying bounds on floating-point roundoff error using PRECiSA [52,33]. PVS includes the NASALib library, which contains formalizations of mathematical concepts in geometry, calculus, real analysis, and quantifier elimination that are useful building blocks for the proofs we present [38].

We automatically generate machine-checkable PVS proof certificates demonstrating correctness for instances of the active corner method. Correspondingly, we have extended the open-source Python implementation of the active corner method to return not only a computationally efficient way to test for collision avoidance, but also a machine-checkable proof certificate of correctness in PVS. These certificates formally demonstrate the correctness of the Python output and bridge the trust gap between the original pen-and-paper proof and the Python code. Note that our certification code automatically generates both the theorem and a corresponding proof script, ensuring fully automatic certification.

The original proof in [28] leveraged significant geometric intuition, which is easily understood but not well suited for formalization in a theorem prover; this work presents a more algebraic approach that is more easily expressed using a proof assistant. We present a novel proof method and proofs of correctness in PVS for certain polygons moving on continuous, piecewise, differentiable trajectories $y = f(x)$. A informal definition of correctness is as follows: when the active

corner method states a point obstacle will be safely avoided, it must indeed be avoided using a more intuitive definition of collision avoidance. Our work can automatically generate PVS proof commands showing the correctness of outputs over regions of the domain in which the trajectory has a bounded derivative, as in the original proof of correctness for the active corner method [28]. We include two case studies intended to demonstrate the applicability of this proof automation in real-world scenarios: the first connects the PVS proofs presented here to the formal embedding of differential dynamic logic (dL) [44] in PVS, Plaidypvs [50], and the second leverages our certificates to show safety for strategic planning and trajectory optimization in air traffic control scenarios.

In brief, the contributions of this work are as follows:

- A novel algebraic proof approach demonstrating correctness for the active corner method.
- A fully automatic certification in PVS implementing that proof approach.
- An extension of the open-source Python implementation of the active corner method to automatically generate PVS-checkable proof certificates.
- A case study extending these results to an embedding of differential dynamic logic (dL) in PVS, showing the equivalence between a dL formula containing differential equations and a quantifier-free equivalent generated by the active corner method.

2 Overview

Consider a polygonal object, representing a bounding volume v moving along some planar trajectory $\mathcal{T}$. The bounding volume v at a point (x, y) in the plane can be denoted $v(x, y)$. Given a single point obstacle in the plane at fixed location (x_O, y_O) , a natural question is whether the object will collide with that obstacle as it moves along its trajectory. An intuitive approach is to simply check all the points along trajectory $\mathcal{T}$, in this work represented as a continuous, differentiable, potentially-piecewise function $f(x)$ in the plane.

Mathematically, that intuitive approach to checking collision avoidance along a trajectory is a *quantified* one: *for all* points $(x_{\mathcal{T}}, y_{\mathcal{T}}) \in \mathbb{R}^2$ along a trajectory $\mathcal{T}$, ensure that an obstacle at coordinate (x_O, y_O) is not inside of a volume v centered at $(x_{\mathcal{T}}, y_{\mathcal{T}}) \in \mathbb{R}^2$. That is,

$$\forall (x_{\mathcal{T}}, y_{\mathcal{T}}) \in \mathcal{T}, (x_O, y_O) \notin v(x_{\mathcal{T}}, y_{\mathcal{T}}). \quad (1)$$

However, this quantified representation has one issue: the $\forall$ quantifier cannot be easily modeled in settings where computational efficiency or complete correctness matters. Simulation-based approaches may rely on imperfect discretization, and the Cylindrical Algebraic Decomposition (CAD) algorithm for eliminating quantifiers is doubly exponential in the number of total variables (not just the quantified variables) [19][18]. Correspondingly, the runtime of quantifier elimination makes CAD intractable even for simple symbolic trajectories that involve only a few parameters. For checking collision avoidance either on-the-fly (at run

time) or at design time, when, say designing an air traffic control system, we need quantifier-free methods of checking collision avoidance.

This work focuses on the certification in PVS of a different approach to finding a collision avoidance statement, one which is free of quantifiers and equivalent to Equation (1). Kheterpal et al. introduced such a fully symbolic method of generating a quantifier-free equivalent to a quantified statement of collision avoidance [28]. The quantifier-free formulation of the “safe region” in that work represented the set of obstacle locations where, given a polygon’s trajectory, a collision will not occur.

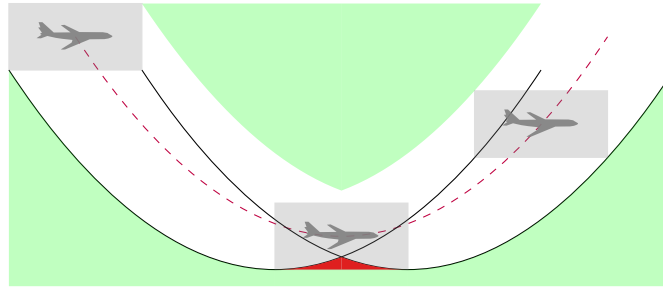


Fig. 1: A rectangular well-clear volume, representing an airplane, moving along a planar trajectory. At the *transition point* at the parabola’s vertex, the “notch” is visible and shaded in red; part of the polygon lies outside of the corner-trajectories at this point.

Consider a running example of the following polygon translating with its center following some trajectory function, displayed in Figure 1:

- An axis-aligned rectangle with width 4 and height 2, so its four corners are at offsets $(\pm 2, \pm 1)$ from the center. In Figure 1, this is the gray rectangle.
- A parabolic trajectory $y = x^2$. In Figure 1, this is the dashed purple line.

The key idea of the active corner method is that, to construct verifiably correct “safe regions” (or reachable sets) for polygonal objects moving in the plane, we must consider the shape of the object **and** the paths of its corners [28]. At any point along the trajectory, two vertices of a convex polygon represent the outermost extent of the polygon as it moves along the trajectory. Those vertices are called *active corners*. For the rectangle under consideration, the bottom-left and top-right corners are *active* for $x \leq 0$, when the trajectory moves downward. For $x \geq 0$, the top-left and bottom-right corners are active. However, only following the paths of active corners could miss the “notch,” an unsound underestimate that occurs when the set of active corners changes; that notch is depicted in Figure 1 in red.

When the trajectory changes direction, the active corners may change as well, as in Figure 1 at the vertex of the parabola. At those instances, called *transition*

points, the boundaries of the safe region follow a side of the polygon; for Figure 1, the boundary follows the bottom edge of the rectangle, thus accounting for the notch. Formally, transition points occur at any point where the tangent line to the trajectory function $f(x)$ is parallel to the slope of an edge of the polygon. In our running example, there is a single transition point at $x = 0$, the vertex of the parabolic trajectory.

In the active corner method, objects are assumed to move without rotating. Because of that, one can shift a (known) trajectory for the object center to get equations of motion for its corners. For the example of a rectangle moving along a parabola given above, the equation of motion for its top-left corner would be $y = (x + 2)^2 + 1$, as the function is shifted left by 2 and up by 1.

The active corner method can be used to evaluate whether a polygon translating along a continuous, differentiable, (potentially) piecewise trajectory $f(x)$ with its center at $(x, f(x))$ will intersect a point obstacle at (x_O, y_O) . If that intersection occurs, the obstacle is deemed unsafe, or equivalently, to lie outside the “safe region” described earlier. To test if an obstacle at (x_O, y_O) is unsafe, the active corner method first checks if the obstacle lies in between the trajectories of the active corners of the polygon, which are opposite vertices for symmetric polygons. That is, an unsafe obstacle must be below the path of one active corner and above the path of its opposing corner. Any obstacle either above or below both corner-trajectories would be avoided, unless it lies in the notch (the solution for this is described below). Since the trajectory of a particular active corner at offset $(\Delta x, \Delta y)$ is $y = f(x - \Delta x) + \Delta y$, one can substitute (x_O, y_O) for (x, y) and check whether y_O is greater than or less than $f(x_O - \Delta x) + \Delta y$ to see which “side” of the corner-trajectory on which the obstacle lies. Equivalently, one can check the sign of $y_O - f(x_O + \Delta x) - \Delta y$. To check this for both active corners and their trajectories, the active corner method checks that the two signs above are **different**; it does not matter which is negative and which is positive.

We can express this check as a product: Equation (2) is a Boolean test for an obstacle at (x_O, y_O) being *unsafe*. Note $\Delta x_i, \Delta x_j, \Delta y_i, \Delta y_j$ are the offsets from the polygon center to the two active corners v_i and v_j , and the check is ≤ 0 because obstacles that lie exactly on a corner-trajectory are indeed unsafe.

$$(y_O - f(x_O + \Delta x_i) - \Delta y_i)(y_O - f(x_O - \Delta x_j) + \Delta y_j) \leq 0 \quad (2)$$

In our running example, the instantiation of Equation (2) yields the following test, which checks if an obstacle (x_O, y_O) lies between **either** pair of active corners (top-left with bottom-right, and bottom-left with top-right):

$$\begin{aligned} &((y_O - (x_O + 2)^2 - 1) \cdot (y_O - (x_O - 2)^2 + 1) \leq 0) \vee \\ &((y_O - (x_O - 2)^2 - 1) \cdot (y_O - (x_O + 2)^2 + 1) \leq 0) \end{aligned} \quad (3)$$

Using Equation (2) is not sufficient; at points along the trajectory where the active corners change, one must account for the *notch* (like the bottom of the parabola in Figure 1). At those discrete instances, called *transition points*, the active corner method additionally checks if an obstacle is unsafe by seeing if it

lies within the polygon at that *transition point*. For this task, several point-in-polygon methods suffice; our proof technique represents an n -sided polygon as the intersection of n inequalities, each of which represents an edge. By checking the disjunction of multiple instances of Equation (2) for every pair of active corners and checking for point-in-polygon at every transition point, the active corner method constructs a test for whether an obstacle at (x_O, y_O) is unsafe.

In our running example, we extend Equation (3) with point-in-polygon checks for an instance of the rectangle centered at $x = 0$, our sole transition point. Note that this representation, the output of the active corner method, includes no trajectory variable x and can be used directly to check whether collision will occur with any point obstacle at (x_O, y_O) . Computationally, only very simple algebra is required.

$$\begin{aligned} & ((y_O - (x_O + 2)^2 - 1) \cdot (y_O - (x_O - 2)^2 + 1) \leq 0) \vee \\ & ((y_O - (x_O - 2)^2 - 1) \cdot (y_O - (x_O + 2)^2 + 1) \leq 0) \vee \\ & ((x_O \geq -2) \wedge (x_O \leq 2) \wedge (y_O \geq -1) \wedge (y_O \leq 1)) \end{aligned} \quad (4)$$

To contrast, the quantified test for collision (the opposite of safety or collision avoidance, hence the use of $\exists$ instead of $\forall$) checks for the existence of some point $(x, y = x^2)$ along the trajectory for which a point obstacle (x_O, y_O) lies inside the boundaries of the rectangle, here represented by the conjunction of inequalities.

$$\begin{aligned} \exists x : & ((x_O - x \geq -2) \wedge (x_O - x \leq 2) \wedge \\ & (y_O - x^2 \geq -1) \wedge (y_O - x^2 \leq 1)) \end{aligned} \quad (5)$$

Prior work by Kheterpal et al. enabled the automatic generation of the quantifier-free expression 4 [28]. In this work, we extend that work to automatically certify in PVS the equivalence of Equations (4) and (5) for specific cases, as described in Section 4.4.

3 A Mechanized Proof in PVS

3.1 Objective

Our soundness goal is to show that any point that the active corner method claims to be safe is indeed safe under the quantified definition in Equation (1).

To achieve this goal, we instead prove the *contrapositive*: that all points $(x_{\text{int}}, y_{\text{int}})$ on the interior of a polygon centered at $(x_{\mathcal{T}}, y_{\mathcal{T}})$ for $y_{\mathcal{T}} = f(x_{\mathcal{T}})$ will evaluate to *unsafe* when substituted in the quantifier-free equivalent test for safety (Equation (2), constructed by shifting the trajectory by active-corner coordinates $(\Delta x_i, \Delta y_i)$ and $(\Delta x_j, \Delta y_j)$).

3.2 Previous Geometric Approach

The original proof of correctness in [28] reasons about segments of a trajectory function in which the active corners do not change; that is, segments where the

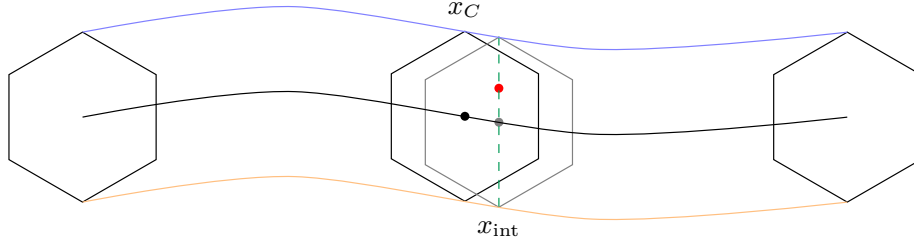


Fig. 2: Shifted polygon illustration. The red point is in the interior of the center black polygon but also lies along the dashed green centerline of the grey polygon.

derivative of the function stays bounded within a certain interval. One key term used to bound the derivative is the slope of a side of the polygon. That proof shows soundness by reasoning about interior points of a polygon and showing that they lie along the center line of a different, shifted polygon (Figure 2). By bounding two quantities:

1. the coordinates of an interior point, and
2. the center coordinates of that shifted polygon,

we can show that the interior point lies on the segment between active corners of the shifted polygon, and so Equation (2) evaluates to True and any interior point will test *unsafe*, as desired.

3.3 Algebraic Approach

We provide a novel proof approach that reasons algebraically rather than geometrically and is more straightforward to prove in PVS, depending only on analysis and algebraic manipulations. For example, it is a challenge to represent the slope of a side of an arbitrary polygon in a proof assistant. We encode the polygon representing the object moving along trajectory $f(x)$ as a conjunction of linear inequalities corresponding to each edge of the polygon. Our proof approach leverages the active corners by combining the inequalities corresponding to the edges adjacent to each corner, in order to show unsafeness according to Equation (2). Figure 3 displays inequalities defining both a square and its active-corner trajectories, for some arbitrary trajectory $f(x)$. For the prior example of a square moving along parabola x^2 , all instances of $f(x)$ would be instantiated with x^2 .

We automatically generate inequalities defining the edges of the polygon (assuming the center is at $(0,0)$) by:

- using the point-slope equation for each pair of adjacent points to get the edge;
- substituting $(0,0)$ to determine the correct direction of the inequality;
- in order to get inequalities describing if (x_O, y_O) is contained in the general polygon centered at $(x, y := f(x))$, substituting $(x_O - x, y_O - f(x))$ for (x, y) .

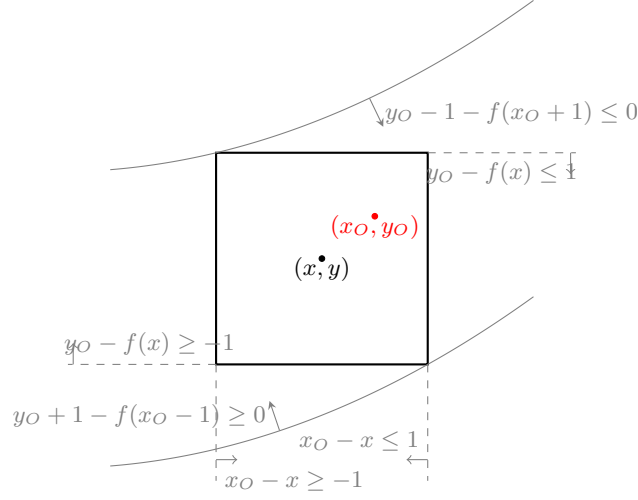


Fig. 3: Square moving along some function $f(x)$. An obstacle (x_O, y_O) is labeled, along with inequalities defining the sides of the square and the active-corner-trajectories.

This last step combines two transformations: we use (x_O, y_O) instead of (x, y) since we are interested in the location of the obstacle, and we subtract $(x, f(x))$ from (x_O, y_O) so inequalities are relative to the polygon center along trajectory $(x, f(x))$ rather than $(0, 0)$.

For instance, in our running example, the inequalities defining the edges of a rectangle are are

$$\begin{aligned}
 y_O - x^2 &\geq -1 \\
 y_O - x^2 &\leq 1 \\
 x_O - x &\geq -2 \\
 x_O - x &\leq 2
 \end{aligned} \tag{6}$$

Figure 3 displays the relevant inequalities for a square.

This method has allowed us to automatically generate our proof goals: PVS lemmas of soundness given a polygon, trajectory, and list of active-corner pairs. That process is described later in Section 4.

In our certification, unlike the original fully symbolic active corner method, we require that the vertices and trajectories be instantiated with real numbers, since the computation is significantly more difficult with symbolic vertices. This limits our certification compared to the fully symbolic nature of the original active corner method, but still permits certification for real-world examples as detailed in Section 6.

In Section 4, we outline the procedure itself and discuss the limitations of automating it in PVS.

3.4 Proof Structure

The proof of the active corner method structures its arguments segment-by-segment along a trajectory, reasoning over domains in which the active corners (defining the outermost extent of the polygon) do not change.

For example, for our running example of a square moving on function $y = x^2$ over the reals, the active corners change (or there is a "transition point") at $x = 0$. Thus there are two "sections" to our proof, generated automatically: one over the domain $(-\infty, 0]$, and another over the domain $[0, \infty)$.

Our PVS certification proceeds similarly. We extend the Python implementation to compute details required to structure our proofs, such as the domain under consideration and any bounds of the derivative in that domain. This was one reason to choose certification of outputs rather than a full formalization of the method in PVS; much of the work done to compute transition points, derivative bounds, and more is straightforward using Python and SymPy [31]. In this way, Python and SymPy are used almost as untrusted oracles, with computation checked in PVS. As the original active corner method implementation was written using SymPy, any limitations of SymPy would affect generation of the quantifier-free result. Therefore, any certificate generated remains valid for showing soundness, as SymPy errors would at worst prevent generating any output and, since certificates are generated using SymPy output, a certificate.

At the end of our proof certificate, we provide a "unifying lemma," uniting the piece-by-piece cases into a single theorem proving soundness and safety throughout the domain under consideration.

3.5 Segment Proofs

Consider a segment of the trajectory $f(x)$ where the pair of active corners does not change and there are no piecewise case switches. Over this segment, our goal is to show that the test in Equation (2) holds for every obstacle point (x_O, y_O) that is inside the polygon when it is centered at $(x, f(x))$. Because the active corners do not change, the slope of the function $f(x)$ is limited; active corners only change when the tangent line to the trajectory is parallel to an edge of the polygon.

For example, for an axis-aligned rectangle, the active corners change when the slope of the trajectory equals zero, which is the slope of the bottom and top edges of the rectangle. Thus, we consider domains over which $f'(x) \geq 0$ and $f'(x) \leq 0$. For an equilateral diamond, the active corners change when the slope of the trajectory equals -1 or 1 . Thus, we would consider domains over which $f'(x) \geq 1$, $f'(x) \in [-1, 1]$, and $f'(x) \leq -1$.

The polygon interior is described by a small set of edge inequalities, as seen in Figure 3. For a certain x , the horizontal offset of any *unsafe* interior point is bounded (the obstacle cannot be arbitrarily far left or right of the center).

The inequalities defining any polygon involve $y_O - f(x)$, as seen in Figure 3, while the test in Equation (2) includes no x , only terms like y_O, x_O , and $f(x_O)$. Our proof technique rests on using the *mean value theorem*: over a region where

the derivative $f'(x)$ is bounded, shifting the evaluation point from x to x_O (or to $x_O \pm n$ for some small n) changes f by an amount we can bound using exactly the bound we have on the derivative $f'(x)$ over domain D . As Equation (2) is a product of two inequalities, our proofs use the mean value theorem twice: once to show that one of the inequalities in Equation (2) is nonpositive and again to show that the other is nonnegative.

3.6 Unifying Lemma

The per-segment proofs described above hold over domains where the active corners do not change, and thus trajectory derivative $f'(x)$ is bounded. Our certificate links together the per-segment proofs into a "unifying lemma" that combines these per-segment proofs into a single theorem showing soundness. For our running example, this lemma would link together the proof cases for $x \leq 0$ (one set of active corners for $f'(x) \leq 0$) and $x \geq 0$ (the other set of corners corresponding to $f'(x) \geq 0$).

4 Automation in PVS

Our certification begins with the open-source Python implementation from the original active corner method paper [28], and extends the Python computation in SymPy to automatically generate proof scripts for PVS. Given a trajectory, domain, and polygon, a Python script generates proof scripts which can be checked, as displayed in Figure 4.

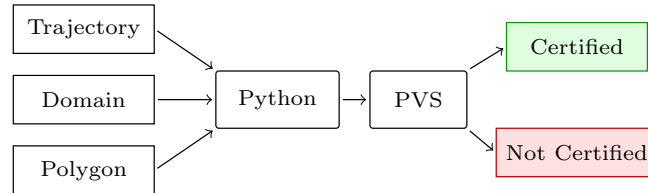


Fig. 4: Proof automation structure.

In order to automatically construct PVS lemmas representing soundness, our certification constructs inequalities that represent each edge of the polygon. The process implements the automatic inequality construction described in Section 3.3.

In order to generate the goal, we run the original active corner method to generate the quantifier-free formulation, which is the disjunction of several products, each of two inequalities. In our proof goals, we restate Equation (2) as a conjunction of inequalities $\geq$ or ≤ 0 , rather than multiplying two inequalities, because the conjunction is an easier proof goal to discharge in PVS. Note that

both possibilities of positive/negative sign are stated in this formulation, so the conjunction of inequalities is equivalent to the product representation.

Our resulting proof goal is to show that Equation (5) implies Equation (4), stated without multiplication:

$$\begin{aligned}
& \exists x : ((x_O - x \geq -2) \wedge (x_O - x \leq 2) \wedge \\
& \quad (y_O - x^2 \geq -1) \wedge (y_O - x^2 \leq 1)) \\
& \quad \implies \\
& ((y_O - (x_O - 2)^2 - 1) \leq 0 \wedge (y_O - (x_O + 2)^2 + 1) \geq 0) \vee \\
& ((y_O - (x_O - 2)^2 - 1) \geq 0 \wedge (y_O - (x_O + 2)^2 + 1) \leq 0) \vee \\
& ((y_O - (x_O + 2)^2 - 1) \leq 0 \wedge (y_O - (x_O - 2)^2 + 1) \geq 0) \vee \\
& ((y_O - (x_O + 2)^2 - 1) \geq 0 \wedge (y_O - (x_O - 2)^2 + 1) \leq 0) \vee \\
& ((x_O \geq -2) \wedge (x_O \leq 2) \wedge (y_O \geq -1) \wedge (y_O \leq 1))
\end{aligned} \tag{7}$$

4.1 Lightweight Proof Automation with Profflite

As a fully automatic method of certifiably checking collision avoidance, we aim for our certificate generator to be accessible to non-formal methods practitioners such as engineers. This guided our choice to leverage a widely-used library and language like SymPy and Python, respectively. Directly integrating the proof certificate generation with the existing active corner implementation makes it easy for any Python user to additionally generate and check certificates with the implementation. Furthermore, because of the segment-by-segment structure of our proof (Section 3.4), any certificate generation would require some amount of computation in Python (or another platform supporting symbolic mathematics), regardless of how the proof steps were specified in PVS. For example, identifying piecewise cases, transition points, and bounds on the trajectory derivative all rely on SymPy symbolic differentiation and numeric evaluation, which would be unnecessarily difficult if done entirely in PVS. This factor was another reason to pursue a certification of the active corner method, rather than a full formalization in PVS.

As such, we chose to build proof certificates in Profflite, which is a lightweight proof automation module within PVS [35]. Profflite scripts allow users to specify sequences of tactics and include them alongside theorems or lemmas. Using Profflite, our soundness proof certificates consisted of a single PVS file both stating properties of soundness and providing their proofs. Better yet, PVS proofs using Profflite scripts can be proven straightforwardly from the command line: using `proveit` will typecheck and prove all lemmas included in a PVS file, reporting which were unproved or not attempted (due to proofs not being specified).

When conducting this work, our proof workflow consisted of iteratively proving more complex theorems of soundness, by modifying domains, polygon dimensions, trajectory functions, and the number of proof cases. Once we had a fully-proved example soundness case, we attempted to streamline the proof as

much as possible at the tactic level; for example, removing extraneous proof commands, hard-coding as few proof terms in the sequent as possible, etc. We then generated a Proffite script for that theorem in PVS automatically, proceeding to parametrize it in Python to generate, say, a similar case but for a different domain or derivative bound. We constructed scripts by computing values for variables in the proof script and filled in a template using Python f-strings. The example in Figure 5 shows part of a Proffite script *template* for one soundness lemma; note that terms in curly braces like `{max_right}` are replaced with terms like `xo - 2` in an actual proof script when instantiated using a Python f-string.

```
%|- (SPREAD (CASE "{case_label1}")
%|- ((THEN (FLATTEN) (LEMMA "{deriv_lemma}")
%|- (SPREAD (INST -1 "f" "{domain_start}" "{domain_end}"
%|- "{deriv_bound}" "{max_right}" "x")
%|- ((SPREAD (SPLIT -1)
%|- ((THEN (ASSERT) (LEMMA "{deriv_lemma}")
%|- (SPREAD (INST -1 "f" "{domain_start}" "{domain_end}"
%|- "{deriv_bound}" "x" "{min_left}")
%|- ((ASSERT) (THEN (EXPAND "ci") (PROPAX)))
%|- (THEN (ASSERT) (EXPAND "ci") (PROPAX))))))
%|- (ASSERT) (PROPAX) (PROPAX) (PROPAX)))
%|- (THEN (EXPAND "ci") (ASSERT)) (THEN (EXPAND "ci") (ASSERT))))
```

Fig. 5: Proffite script template example, displayed here for a closed interval ("`ci`") subdomain.

4.2 Segment Proof Automation

To prove the soundness lemma for each segment of the proof certificate, in which active corners do not change (and thus the trajectory derivative can be bounded), we use two known terms: the edge-inequalities that define the polygon and the derivative bounds over that segment/domain. Next, we introduce the mean value theorem lemma corresponding to that segment. With these pieces in place, we can relate $f(x)$ and $f(x_O)$ to prove the consequent.

Using the `assert` command allows us to automatically simplify expressions and propositions for particular cases [49]. This command simplifies using decision procedures and applies a number of automatic rewrites; it can prove theorems that involve only linear inequalities and propositional logic, such as some of our proof cases. In our proofs, we use `assert` after stating and instantiating the relevant mean value theorem lemma, letting us show part of or a full term in the consequent, in combination with terms like the derivative bounds (since the active corners do not change) and the edge-inequalities that define the polygon.

Consider, for example, the ascending segment of our rectangle on parabola running example. In this case, the derivative bound is $f'(x) \geq 0$ for domain $x \geq 0$. Correspondingly, the proof script for this segment calls a mean-value theorem lemma corresponding to 1) a lower bound on the derivative over 2) a domain unbounded to the right. The proof script then instantiates that lemma with our domain single-side bound 0, derivative bound 0, trajectory variable x ,

```

1: ge_ro_case_1: LEMMA
2:   FORALL(xo,yo:real, g:[real -> real]):
3:     LET f = LAMBDA(x:real):
4:       COND
5:         x < 0 -> g(0),
6:         x >= 0 -> g(x)
7:       ENDCOND
8:   IN
9:   derivable?[(right_open(0))](f) AND
10:  (FORALL(x:(right_open(0))):
11:    deriv[(right_open(0))](f)(x) >= 0)
12:  AND (EXISTS (x : real) :
13:    ((-x + xo + 2 >= 0) AND
14:     (-x + xo - 2 <= 0) AND
15:     (yo - f(x) + 1 >= 0) AND
16:     (yo - f(x) - 1 <= 0)) AND
17:    x >= 0 AND
18:    xo + 2 >= x AND xo + -2 <= x)
19:  IMPLIES
20:  ((yo - f(xo - 2) - 1 >= 0) AND
21:   (yo - f(xo + 2) + 1 <= 0)) OR
22:  ((yo - f(xo - 2) + 1 >= 0) AND
23:   (yo - f(xo + 2) - 1 <= 0)) OR
24:  ((yo - f(xo + 2) - 1 >= 0) AND
25:   (yo - f(xo - 2) + 1 <= 0)) OR
26:  ((yo - f(xo + 2) + 1 >= 0) AND
27:   (yo - f(xo - 2) - 1 <= 0)) OR
28:  (xo + 2 >= 0) AND
29:  (xo - 2 <= 0) AND
30:  (yo - f(0) + 1 >= 0) AND
31:  (yo - f(0) - 1 <= 0)

```

Fig. 6: Example single-segment lemma for a rectangle of width 4 and height 2. In the antecedent, lines 3-7 define the “clipped” trajectory, 9-11 bound the derivative, 12 states some unsafe x exists, 13-16 define the polygon edges, 17-18 restrict the domain. In the consequent, lines 20-27 state the quantifier-free tests from Equation 2, and 28-31 test for the notch. [\[Nishant: move this to artifact or appendix?\]](#)

and the desired term from the proof goal $x_O - 2$. An example of the lemma showing correctness for this segment can be seen in Figure 6.

Our soundness lemmas do split on one or two cases, depending on the domain. This is because of a clipping rule in the original active corner method paper [28, Section III.D], which holds functions constant outside of the domain of a given piecewise function. The rule was originally intended to guard against arbitrary behavior of piecewise trajectories outside their defined subdomain. However, while proving soundness for certain cases, we discovered this “clipping” rule was necessary even for non-piecewise functions. Without clipping, the proof assistant generated cases corresponding to counterexamples in which trajectories curved upwards and valid obstacles were on the “same side” of both active corner trajectories. Figure 7 shows the counterexamples and how clipping the trajectory function to remain constant eliminates these counterexamples. Our discovery of such counterexamples, as well as the more general need for clipping, further highlights the value of formal methods and certification.

One benefit of this approach is that relatively few proof script templates are required when automating proofs. We can simply instantiate the template with the correct arguments (the particular domain and derivative bounds, domain type, polygon dimensions, and so on) to construct a valid proof script.

4.3 Unifying Lemma Automation

In order to prove the unifying lemma, the proof script divides the domain into multiple cases and invokes each of the per-segment lemmas. To show the bounds on the derivative hold for the trajectory in question, we use the `deriv` command in PVS, which differentiates the trajectory function; the limitations of that command are discussed in Section 4.4. The proof proceeds by stating that the domains of the lemma and the case are equivalent and expanding the definitions of the “clipped” functions described above for each per-segment lemma. A variety of side conditions are generated but are straightforward to prove, requiring only a few simple tactics.

In practice, given:

1. a polygon,
2. a trajectory, and
3. a domain containing n intervals over which the active corners do not change:

a total of n per-segment lemmas and one unifying lemma are generated. Despite this, Figure 8 shows many more than a handful of lemmas are attempted and proved. This is for two reasons:

1. The ProofLite certificate file is self-contained, so it includes several theorems defining mathematical properties we need (derived from the mean value theorem for a variety of cases). These lemmas have been fully proved and will be added to NASALib. After that point, their direct inclusion will no longer be necessary in the certificate, so the number of lemmas per example will decrease.

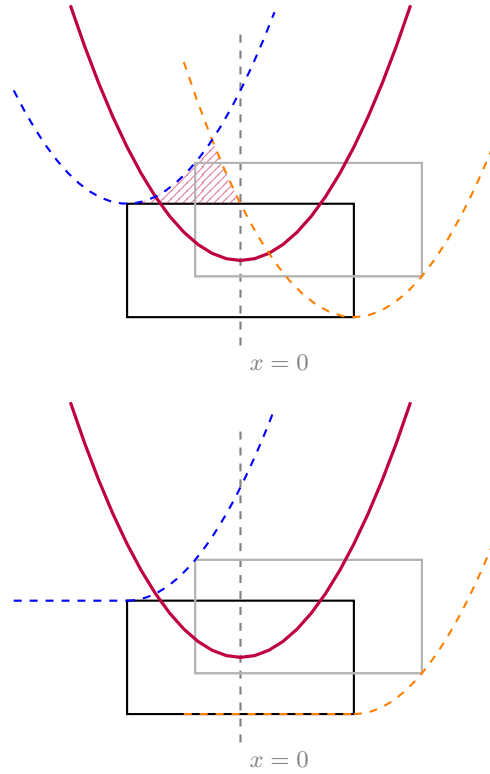


Fig. 7: Proof counterexample necessitating clipping. In the top image, the shaded region is clearly unsafe but is on the same side (below) both the orange and blue corner-trajectories. After the clipping rule was applied in the lower image, the counterexample region no longer appears.

```
Processing certificate.pvs.
Writing output to file certificate.summary
Logging PVS information in certificate.log
Including proof traces in output file
```

```
Proof summary for theory
    active_corner_certificate
Theory totals:
  40 formulas, 40 attempted, 40 succeeded
  (0.01 s)
```

Fig. 8: proveit output from an example certificate

2. PVS automatically generates type correctness conditions, called TCCs, while type-checking. Many of these proofs are easy or even trivial to discharge. In our proof automation, we used approaches including calling lemmas to show differentiability or that a domain is not a singleton, running `assert` when trivial, or running PVS’s automatic `grind` command which repeatedly applies rewrites, simplifications, and more.

4.4 Capabilities and Limitations

We have focused on fully-automatic proof certificate generation in this work, in the spirit of the original active corner method paper [28]. While our code to generate the lemmas in the certificate (both for the per-segment proofs and the unifying lemma) generalizes broadly to convex polygons and a variety of trajectories, the proof automation portion has limitations. At the moment, the proof automation holds only for rectangles: we have focused on this case because rectangular bounding volumes are commonly used in real-world collision detection. Handling arbitrary convex polygons will be the focus of an extension to this work. Additionally, the certification we provide has similar limitations to that of the original active corner method: only convex polygons may be considered, and trajectories must contain a finite number of transition points over the domain under consideration. For example, a rectangle moving along trajectory $f(x) = \sin(x)$ over an infinite domain will have infinite transition points (at the top and bottom of each sinusoid), but a finite number over a finite domain. However, a more poorly-behaved function such as $f(x) = \sin(\frac{1}{x})$ (extended with 0 at $x = 0$), which oscillates infinitely as it approaches $x = 0$, cannot be handled in any domain containing 0. Vertical trajectories cannot be expressed using functions $f(x)$; in these circumstances, a change of coordinates to express functions $x = f(y)$ instead as $y = f(x)$ in a rotated frame will allow us to handle such trajectories.

Our proof automation of the unifying lemma is limited to trajectories and domains for which there are less than three piecewise cases and/or “segments” over which the active corners do not change. As the unifying lemma proofs seek to join together multiple smaller proof cases, they assume the trajectory is defined over all reals, not a smaller domain. (Note that cases where the domain is a subset of the reals are subsumed by the broader domain of all $\mathbb{R}$.) Construction of the Proffite script for our unifying lemmas themselves is tedious, as each unifying lemma may invoke an arbitrary number of per-segment lemmas. Automating the construction of that proof is a goal for future work and requires building a more robust proof tree generator.

Additionally, the unifying lemmas rely on the `deriv` command to differentiate trajectory functions and show they obey the derivative bounds in the correctness lemmas. That `deriv` command is not guaranteed to succeed in a fully automatic proof, as it relies on more complex calculus functions in PVS. That said, our proof certificates include a more general, but more robust “helper” lemma that simply assumes the SymPy-calculated derivative bounds hold without differentiating any particular function. Our proofs employ a two-part structure, first proving

that helper lemma and then showing the trajectory function indeed obeys those derivative bounds to prove the strongest unifying lemma.

5 Examples

Our supplementary material contains our Python implementation to automatically generate correctness certificates for the active corner methods, proof scripts for our case studies, and a selection of example certificates. These examples are described in the following section. All certificate examples begin with Python files describing the three inputs to our certification:

1. A (potentially-)piecewise trajectory function $f(x)$.
2. A domain, either unbounded or bounded over real numbers.
3. A polygon; as described in Section 4.4 our fully automatic proofscript generation supports only rectangles but lemmas can be generated for other convex polygons.

The `README.md` file in supplementary material describes how to automatically generate and then check a proof certificate given a Python and SymPy specification. In the following examples, rectangles of arbitrary numeric dimensions (including squares) are supported.

(E1) Linear ascent on a closed interval. Let $f(x) = x$ on $[-10, 10]$. There is a single segment with $f'(x) = 1 \geq 0$. The certificate comprises one segment lemma; the unifying lemma leverages the one segment lemma.

(E2) Linear descent on the reals. Let $f(x) = -x$ on $\mathbb{R}$. There is a single (unbounded) segment with $f'(x) = -1 \leq 0$. No clipping (as displayed in Figure 7) is required; the unifying lemma discharges straightforwardly from the segment lemma without needing to split into cases.

(E3, E4) Parabolas on the reals. For $f(x) = x^2 - 4x + 3$ and $f(x) = -x^2 - 10x - 25$, each trajectory has one transition point at the parabola's vertex, yielding two segments. For the first parabola, this vertex is at $x = 2$, yielding segments $(-\infty, 2]$ and $[2, \infty)$; the second has a vertex at $x = -5$ and segments $(-\infty, -5]$ and $[-5, \infty)$. The certificates for each example lay out two segment lemmas plus a unifying lemma over $\mathbb{R}$ that bridges the vertex transition. The clipping rule illustrated in Figure 7 is used in both these and the following examples.

(E5, E6, E7) Parabola-linear hybrids (single transition at the vertex). We include two piecewise, continuous, differentiable trajectories, representing shifted versions of each other:

$$f(x) = \begin{cases} x^2, & x \leq 4, \\ 8 * x - 16, & x > 4, \end{cases}$$

$$f(x) = \begin{cases} x^2 + 2x + 1, & x \leq 3, \\ 8x - 8, & x > 3. \end{cases}$$

Each has one transition point (at $x = 0$ and $x = -1$, respectively), so the proof consists of two segment lemmas (left/right of the vertex) and a unifying lemma stitches the cases together. **Note:** in PVS, the `deriv` tactic is more robust on expanded polynomials; expanding $(x + 1)^2$ as $x^2 + 2x + 1$ improves automation when discharging the derivative-bound obligations. Our examples include both the factored and expanded forms to illustrate this; the final unifying lemma fails to prove fully if $(x + 1)^2$ is used but succeeds for $x^2 + 2x + 1$.

[Nishant: Talk about the times here]

Example	Gen. (s)	Check (s)	Formulas	Status
1	0.69	9.64	34/34	Pass
2	0.50	7.29	31/31	Pass
3	0.54	30.20	42/42	Pass
4	0.54	34.10	42/42	Pass
5	0.69	4.06	—	Fail
6	0.65	3.92	—	Fail
7	0.64	3.88	—	Fail

Table 1: Proof certificate generation and checking times.

6 Case Study: Differential Dynamic Logic

Using Plaidypvs, the formal embedding of `dL` into PVS [50], we examine a case study inspired by Jeannin et al. [25]. This paper generated quantifier-free safe regions for an aircraft collision avoidance system, ACAS X. We a slightly modified version of a `dL` collision avoidance property relating to air traffic control, stated in `dL` using differential equations to govern motion, and provably certifies the equivalence to a quantifier-free safe region using Plaidypvs, the active corner method, and our proof approach. The definition of safety was provided by a Differential Dynamic Logic (`dL`) formula [44], which quantified over space and time. The underlying differential equations were solved by hand and quantifier-free safe regions manually generated, which took on the order of hundreds of person-hours. The equivalence between the `dL` representation and quantifier-free safe regions was proved in KeYmaera X [45]. The example we present here is simplified from past hybrid programs, as the original `dL` formula reasoned over time t and bounded aircraft trajectories using upper or lower bounds. Equation 8 presents a `dL` formula that specifies collision avoidance for a point obstacle at (r_O, h_O) for a rectangular bounding volume representing an aircraft. The aircraft begins by at a zero climb rate ($v = 0$) but accelerates upward at a constant rate

(symbolic variable a); the resulting trajectory is parabolic with curvature defined by symbol a . In practice, the motion described by this **dL** program reduces to the right half of Figure 1.

As in Jeannin et al., we use the same reference frame to reduce the three-dimensional statement of collision avoidance to a two-dimensional one. The encounter is modeled as a one-dimensional vertical encounter along with the distance in the horizontal axis [24, Section 6].

6.1 Simple **dL** example

The **dL** statement of collision avoidance is as follows:

$$\begin{aligned}
 & r_p > 0 \wedge h_p > 0 \wedge r = 0 \wedge h = 0 \wedge \\
 & v = 0 \wedge r_v > 0 \wedge a > 0 \rightarrow \\
 & [(\{r' = r_v, h' = v, v' = a \\
 & \quad *)((r - r_O \geq -r_p) \wedge (r - r_O \leq r_p) \wedge \\
 & \quad (h_O - h \leq h_p) \wedge (h_O - h \geq -h_p))]
 \end{aligned} \tag{8}$$

This formula states that a rectangular bounding volume with positive radius r_p and height h_p moves with a constant horizontal velocity (r') of r_v and accelerates upward (v') at a constant rate a . The postcondition at the end formalizes collision avoidance: the aircraft will not collide with a point obstacle at r_o, h_o .

6.2 Formalization in PVS

PVS artifacts for this case study are available in supplementary material. The PVS lemmas and proof scripts for this case study are not automatically generated, so no Python example is provided. The **dL** formula in Equation 8 was expressed in Plaidypvs, which enabled a conversion to an equivalent native PVS expression of the same statement, though r_p, h_p were instantiated with numeric values. The **dL-solve** tactic included in Plaidypvs solves the differential equations describing the trajectory and reduces the collision avoidance statement to inequalities that correspond to a rectangular bounding volume. Note that this proof leaves acceleration $v' = a$ as some positive variable, defining the curvature of the parabolic trajectory. In future work, we look forward to more deeply supporting symbolic trajectory variables for fully automatic proof certificate generation.

From there, we can apply the active corner proof automation to show the equivalence between the **dL** statement and the quantifier-free safe regions. This case study is provided in our artifact as a PVS file, with accompanying proof. An extension to a full general proof automation for **dL** formulas is the focus of future work. Note that in the work of Jeannin et al., the model of aircraft begins by descending but accelerates upward at a constant rate, until it reaches a target climb rate; the resulting trajectory is piecewise parabolic for the acceleration and then linear upon reaching the target climb rate [25]. The case study

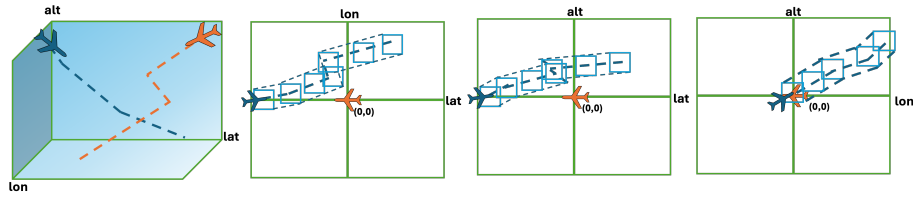


Fig. 9: This figure visualizes the computation of piecewise relative trajectories between two aircraft in four dimensions (latitude, longitude, altitude, and time). **Far Left:** A 3-D view of the two flight paths, with one aircraft shown as a dashed blue line and the other as a dashed orange line. **Middle Left:** The Lat–Lon Relative Trajectory depicts horizontal separation. The blue curve traces the position of one aircraft relative to the other (fixed at the origin), highlighting changes in lateral and longitudinal distance over time. **Middle Right:** The Alt–Lat Relative Trajectory shows the corresponding altitude–latitude relationship. **Far Right:** The Alt–Lon Relative Trajectory presents the altitude–longitude relationship. Each panel reduces the complex 4-D relative motion to a simpler 2-D projection, using a series of squares to indicate the safety cube around the aircraft.

we expressed in PVS can be extended to this more complex hybrid case with piecewise behavior, but would require further development to Plaidypvs. That extension is left as a future verification effort.

7 Case Study: Strategic Planning for Air Traffic Control

Our second case study concerns the strategic planning of airspace, where air traffic control plans the trajectories of aircraft entering an airspace with the intention to land at a given airport. This problem setting is particularly relevant for trajectory optimization, where an optimizer outputs discrete paths defined by waypoints for each aircraft. However, after the optimization is performed, it is necessary to verify that the resulting trajectories remain collision-free. In this case study, we formally validate one set of optimization results against a predefined safety specification using the certificate generation implementation in this paper.

7.1 Simple Two-Aircraft Planning

We study two optimized aircraft trajectories in four dimensions (latitude, longitude, altitude, and time). The trajectories were generated by a Mixed-Integer Linear Programming (MILP) optimizer, which takes as input the historical pre-Terminal Radar Approach Control (TRACON) entry points of the two aircraft and computes flight paths that satisfy required safety separations while minimizing fuel consumption. The trajectories of the two aircraft, output by the optimization, is a series of timed waypoints.

To enable verification, we first generate the *relative* trajectory of aircraft A with respect to aircraft B , treating B as a point obstacle under the assumption of static environments: concretely, we consider the trajectory of aircraft A in the referential of aircraft B . We consider a well-clear volume around aircraft A of side 1,000 ft in latitude, longitude and height (± 500 ft). Figure 9 displays how the relative trajectories are generated. We then construct a piecewise linear function to approximate motion between consecutive waypoints using linear interpolation, assuming pilots follow them exactly. In this first-order approximation, each segment of the trajectory is represented by a straight line connecting successive waypoints. To illustrate, consider a two-dimensional relative trajectory defined by waypoints $(x_0, y_0), \dots, (x_n, y_n)$, where n is the number of waypoints. Since this is a relative trajectory, we do not need timestamps anymore to define collision avoidance. Also note that given the cubic nature of the well-clear volume in this example, absence of collision in any two dimensions (latitude-longitude, latitude-altitude or longitude-altitude) implies absence of collision in three dimensions. The corresponding linear piecewise function is:

$$f(x) = \begin{cases} m_0x + b_0, & x \geq x_0 \text{ \& } x \leq x_1, \\ \dots & \\ m_nx + b_n, & x \geq x_{n-1} \text{ \& } x \leq x_n. \end{cases}$$

where $\forall i \in [0, n-1]$, $m_i = \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$ and $b_i = y_i - m_i x_i$, assuming x is a unique set of values.

Then, using the generated piecewise function and the required safety cube, we apply the automatic certificate generator implementation and prove that the quantifier-free safe region is equivalent to the relative trajectory. As a result, we are able to validate our safety condition against the verified quantifier-free boolean condition for the unsafe region. This case study is provided in our artifact as a PVS file, with its accompanying proof. The PVS lemmas and proof scripts for this case study are not automatically generated, so no Python example is provided. That said, our automatic certificate generation scripts were useful in building an initial draft specification and proof script, which was manually modified for the final version.

8 Related Work

PVS has been used to verify properties of hybrid systems in the past [51,2] along with tools like Isabelle [21]. Barendregt and Barendsen [11] lay out various approaches for trusting algorithms: an *autarkic* approach, where an algorithm is fully reimplemented in a theorem prover, or a *skeptical* approach, where a “trace” (laying out the steps taken) of computation or proof is verified. Our certificate structure, and many others, follows this *skeptical* approach. Our work in generating checkable proof certificates is reminiscent of the Proof Module in CVC [27,9]. CVC5 uses a domain-specific language to generate these proofs

in a modular, lazily evaluated fashion [41,10]. Other satisfiability (SAT) and satisfiability modulo theories (SMT) solvers include certificate capabilities, such as Z3 [14,13], VeriT [48] and zChaff (which uses Rocq) [26].

PVS has a long history of being applied to the formal verification of safety-critical systems, particularly in the domain of air traffic management and unmanned aircraft systems (UAS). In the context of collision avoidance, PVS has been used to model and verify early aircraft separation systems [15], and more recently to develop and analyze advanced UAS safety concepts such as PolySafe [16]. PVS played a central role in the formal development of DAIDALUS [34], the reference implementation of detect-and-avoid (DAA) algorithms, included in Appendix G of RTCA/FAA DO-365 [1], standards that represent a step toward the trustworthy deployment of autonomous collision-avoidance technologies in the national airspace. Our case study presented in Section 6 is based on work done to verify ACAS X by Jeannin et al. [24,25], for which verification was performed in Key-Maera X [45]. A comparison between ACAS Xu, a related system to ACAS X [30], and DAIDALUS has been conducted [20].

Due to the safety-critical nature of the task, many methods for collision avoidance checking exist. Zonotope reachability methods iteratively propagate a set-based abstraction to compute an estimate of safety [23], and have been used to verify safety for automated road vehicles [4] and quadrotor aircraft [29]. Barrier certificates are a common tool for verifying safety invariants for dynamical and hybrid systems [46], as has Hamilton-Jacobi reachability [8]. Safe controllers can be synthesized using control barrier functions [6].

Beyond collision avoidance, PVS has supported a range of mathematical foundations necessary for certification of safety-critical systems. This includes libraries for real analysis, hybrid systems reasoning, and decision procedures based on Sturm’s and Tarski’s theorems for univariate quantifier elimination [37]. PVS itself can export externally-checkable proof certificates [22] to Dedukti [47] and MetiTarski [3]. Certification-inspired methods have also been explored in the domain of numerical software, most notably through PRECiSA, a tool for statically certifying bounds on floating-point roundoff error using PVS proofs [52,33]. Similar finite-precision error bounds can be formalized using in Rocq and HOL4 [12]. Armand et al. presented proof witnesses for SMT solving in Rocq [7]. Proof-carrying code implementations also rely on a *certifier* to check correctness of compilation and computation, with early work in the field done by Necula et al. [39,17].

9 Reflection and Conclusion

We have presented work on certifying the active corner method [28]. Our goal is to generate machine-checkable proof certificates of correctness and provide these to users of the extended active corner method implementation, thus bridging the trust gap between the active corner method’s original pen-and-paper proof and the provided Python implementation. In this paper, we have reviewed the active corner method; discussed two proof approaches: the original geometric

proof from [28] and a novel, more algebraic approach; presented progress on formalizing this new proof in PVS; presented our automatic proof certificate generator; and discussed several fully-automatic examples and two case studies requiring some manual proof effort.

Among the benefits of using PVS were the volume of mathematical formalizations already completed. For example, the mean value theorem and related differentiation concepts were already formalized, providing the mathematical backbone for our bounding arguments using the derivative. Beyond the uses of this particular certification of the active corner method, we plan to contribute several already-proven formalizations to NASALib, such as the extensions to the mean value theorem cases needed for our certification. These were developed for our proofs when seeking to bound the differences between $f(x)$ and $f(x_O)$ using the bounds on trajectory derivatives.

We hope to encourage further use of ProofLite as a more accessible proof automation platform. Section 4.1 describes our workflow for parametrizing and automatically constructing ProofLite scripts for our soundness proofs. Our iterative workflow to generalize proof scripts may be applicable for certification and formalization tasks when full proof strategy development proves unnecessary. That said, for proofs that are more complex, we plan to go further and develop a more thorough proof tree generator to automate the construction of the proof script. Generating such a bank of example proofs may be useful in other ways when automating proofs, in particular when integrated with large language models (LLMs). LLMs may be able to successfully automate routine proof steps and suggest proof strategies based on similar previously solved problems; some steps in our proof generalization workflow could have been streamlined using LLM assistants like CoProver [53].

There were a few drawbacks to the use of PVS for this certification. Because of the lack of a complete formal geometry library, we had to develop a new, fully algebraic proof approach. Our existing proof strategies are somewhat limited: the ease of use of ProofLite is, unsurprisingly, restricted in flexibility compared to the expressive power of proof strategies written in Lisp or wholly new PVS commands. An additional step to support greater proof automation would be adding more mathematical primitives to PVS; the existence of calculus & mean-value theorem concepts as well as real analysis formalisms like domains was essential in conducting this certification. However, formalized notions of geometry such as polygon definitions, manipulations like computing vertices and edges, as well as stronger automatic derivative reasoning would have streamlined this work even further.

We look forward to several avenues of future work:

- Extending our proof certificates to handle more complex classes of trajectories & polygons.
- Supporting symbolic trajectory parameters in automatic proof certificate generation.
- A more robust proof tree generator for unifying lemmas.

- Further support for hybrid programs representing piecewise trajectories in Plaidypvs, towards supporting proof automation for the full case study described in Section 6. We envision extending the PVS-dL embedding and leveraging the active corners certification to automate the verification process conducted by Jeannin et al. [25].
- A study of the performance impacts of generating soundness proofs in addition to generating quantifier-free safe region formulations, as well as an understanding of the computational complexity of our proof cases.
- Contributing further relevant mathematical formalizations to NASALib.

Acknowledgments

This work was funded by NASA Fellowship #80NSSC20K1465. Thank you to Aaron Dutle, Cesar Muñoz, and Mariano Moscato for their assistance in developing these proof techniques and the PVS proofs.

References

1. 228, R.S.C.: Do-365: Minimum operational performance standards (mops) for detect and avoid (daa) systems. Tech. rep., RTCA, Inc. (2017), <https://standards.globalspec.com/std/14562761/do-365c/>
2. Abraham-Mumm, E., Hannemann, U., Steffen, M.: Verification of hybrid systems: Formalization and proof rules in pvs. In: Proceedings Seventh IEEE International Conference on Engineering of Complex Computer Systems. pp. 48–57. IEEE (2001)
3. Akbarpour, B., Paulson, L.C.: Metitarski: An automatic theorem prover for real-valued special functions. *Journal of Automated Reasoning* **44**(3), 175–205 (2010)
4. Althoff, M., Dolan, J.M.: Online verification of automated road vehicles using reachability analysis. *IEEE Transactions on Robotics* **30**(4), 903–918 (2014)
5. Althoff, M., Frehse, G., Girard, A.: Set propagation techniques for reachability analysis. *Annual Review of Control, Robotics, and Autonomous Systems* **4**, 369–395 (2021)
6. Ames, A.D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., Tabuada, P.: Control barrier functions: Theory and applications. In: 2019 18th European Control Conference (ECC). pp. 3420–3431. IEEE (2019)
7. Armand, M., Faure, G., Grégoire, B., Keller, C., Théry, L., Werner, B.: A modular integration of sat/smt solvers to coq through proof witnesses. In: International Conference on Certified Programs and Proofs. pp. 135–150. Springer (2011)
8. Bansal, S., Chen, M., Herbert, S., Tomlin, C.J.: Hamilton-jacobi reachability: A brief overview and recent advances. In: 2017 IEEE 56th Annual Conference on Decision and Control (CDC). pp. 2242–2253. IEEE (2017)
9. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) Tools and Algorithms for the Construction and Analysis of Systems - 28th International Conference, TACAS 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings, Part

- I. Lecture Notes in Computer Science, vol. 13243, pp. 415–442. Springer (2022). https://doi.org/10.1007/978-3-030-99524-9_24, https://doi.org/10.1007/978-3-030-99524-9_24
10. Barbosa, H., Reynolds, A., Kremer, G., Lachnitt, H., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Viswanathan, A., Viteri, S., et al.: Flexible proof production in an industrial-strength smt solver. *Automated Reasoning* p. 15 (2022)
11. Barendregt, H., Barendsen, E.: Autarkic computations in formal proofs. *Journal of Automated Reasoning* **28**(3), 321–336 (2002)
12. Becker, H., Zyuzin, N., Monat, R., Darulova, E., Myreen, M.O., Fox, A.: A verified certificate checker for finite-precision error bounds in Coq and HOL4. In: *Formal Methods in Computer Aided Design (FMCAD)*. IEEE (2018). <https://doi.org/10.23919/FMCAD.2018.8603019>, <https://cakeml.org/fmcad18.pdf>
13. Böhme, S., Fox, A.C., Sewell, T., Weber, T.: Reconstruction of z3’s bit-vector proofs in hol4 and isabelle/hol. In: *International Conference on Certified Programs and Proofs*. pp. 183–198. Springer (2011)
14. Böhme, S., Weber, T.: Fast lcf-style proof reconstruction for z3. In: *International Conference on Interactive Theorem Proving*. pp. 179–194. Springer (2010)
15. Carreno, V., Muñoz, C.: Aircraft trajectory modeling and alerting algorithm verification. In: *International Conference on Theorem Proving in Higher Order Logics*. pp. 90–105. Springer (2000)
16. Colbert, B.K., Slagel, J.T., Crespo, L.G., Balachandran, S., Muñoz, C.: Polysafe: a formally verified algorithm for conflict detection on a polynomial airspace. *IFAC-PapersOnLine* **53**(2), 15615–15620 (2020)
17. Colby, C., Lee, P., Nacula, G.C., Blau, F., Plesko, M., Cline, K.: A certifying compiler for java. In: *Proceedings of the ACM SIGPLAN 2000 Conference on Programming Language Design and Implementation*. p. 95–107. PLDI ’00, Association for Computing Machinery, New York, NY, USA (2000). <https://doi.org/10.1145/349299.349315>, <https://doi.org/10.1145/349299.349315>
18. Collins, G.E.: Quantifier elimination for real closed fields by cylindrical algebraic decomposition. In: Brakhage, H. (ed.) *Automata Theory and Formal Languages*. pp. 134–183. Springer Berlin Heidelberg, Berlin, Heidelberg (1975)
19. Davenport, J.H., Heintz, J.: Real quantifier elimination is doubly exponential. *Journal of Symbolic Computation* **5**(1), 29–35 (1988). [https://doi.org/https://doi.org/10.1016/S0747-7171\(88\)80004-X](https://doi.org/https://doi.org/10.1016/S0747-7171(88)80004-X), <https://www.sciencedirect.com/science/article/pii/S074771718880004X>
20. Davies, J.T., Wu, M.G.: Comparative analysis of acas-xu and daidalus detect-and-avoid systems. *Tech. rep.* (2018)
21. Foster, S., Huerta y Munive, J.J., Gleirscher, M., Struth, G.: Hybrid systems verification with isabelle/hol: Simpler syntax, better models, faster proofs. In: *International Symposium on Formal Methods*. pp. 367–386. Springer (2021)
22. Gilbert, F.: Proof certificates in pvs. In: *International Conference on Interactive Theorem Proving*. pp. 262–268. Springer (2017)
23. Girard, A.: Reachability of uncertain linear systems using zonotopes. In: *International Workshop on Hybrid Systems: Computation and Control*. pp. 291–305. Springer (2005)
24. Jeannin, J.B., Ghorbal, K., Kouskoulas, Y., Gardner, R., Schmidt, A., Zawadzki, E., Platzer, A.: A formally verified hybrid system for the next-generation airborne collision avoidance system. In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. pp. 21–36. Springer (2015)

25. Jeannin, J., Ghorbal, K., Kouskoulas, Y., Schmidt, A., Gardner, R.W., Mitsch, S., Platzer, A.: A formally verified hybrid system for safe advisories in the next-generation airborne collision avoidance system. *Int. J. Softw. Tools Technol. Transf.* **19**(6), 717–741 (2017). <https://doi.org/10.1007/s10009-016-0434-1>, <https://doi.org/10.1007/s10009-016-0434-1>
26. Jouannaud, J.P., Strub, P.Y., Zhang, L.: Certification of sat solvers in coq. In: *Guangzhou Symposium on Satisfiability in Logic-Based Modeling* (2010)
27. Katz, G., Barrett, C., Tinelli, C., Reynolds, A., Hadarean, L.: Lazy proofs for dpll (t)-based smt solvers. In: *2016 Formal Methods in Computer-Aided Design (FMCAD)*. pp. 93–100. IEEE (2016)
28. Kheterpal, N., Tang, E., Jeannin, J.B.: Automating geometric proofs of collision avoidance with active corners. In: Griggio, A., Rungta, N. (eds.) *Proceedings of the 22nd Conference on Formal Methods in Computer-Aided Design – FMCAD 2022*. vol. 3, pp. 359–368. TU Wien Academic Press (2022)
29. Kousik, S., Holmes, P., Vasudevan, R.: Safe, aggressive quadrotor flight via reachability-based trajectory design. In: *Dynamic Systems and Control Conference*. vol. 59162, p. V003T19A010. American Society of Mechanical Engineers (2019)
30. Manfredi, G., Jestin, Y.: An introduction to acas xu and the challenges ahead. In: *2016 IEEE/AIAA 35th Digital Avionics Systems Conference (DASC)*. pp. 1–9. IEEE (2016)
31. Meurer, A., Smith, C.P., Paprocki, M., Čertík, O., Kirpichev, S.B., Rocklin, M., Kumar, A., Ivanov, S., Moore, J.K., Singh, S., Rathnayake, T., Vig, S., Granger, B.E., Muller, R.P., Bonazzi, F., Gupta, H., Vats, S., Johansson, F., Pedregosa, F., Curry, M.J., Terrel, A.R., Roučka, v., Saboo, A., Fernando, I., Kulal, S., Cimrman, R., Scopatz, A.: Sympy: symbolic computing in python. *PeerJ Computer Science* **3**, e103 (Jan 2017). <https://doi.org/10.7717/peerj-cs.103>, <https://doi.org/10.7717/peerj-cs.103>
32. Mitsch, S., Ghorbal, K., Platzer, A.: On provably safe obstacle avoidance for autonomous robotic ground vehicles. In: *Robotics: Science and Systems IX*, Technische Universität Berlin, Berlin, Germany, June 24–June 28, 2013 (2013)
33. Moscato, M., Titolo, L., Dutle, A., Munoz, C.A.: Automatic estimation of verified floating-point round-off errors via static analysis. In: *International Conference on Computer Safety, Reliability, and Security*. pp. 213–229. Springer (2017)
34. Muñoz, C., Narkawicz, A., Hagen, G., Upchurch, J., Dutle, A., Consiglio, M., Chamberlain, J.: Daidalus: detect and avoid alerting logic for unmanned systems. In: *2015 IEEE/AIAA 34th Digital Avionics Systems Conference (DASC)*. pp. 5A1–1. IEEE (2015)
35. Munoz, C.A.: Batch proving and proof scripting in pvs. *Tech. rep.* (2007)
36. Muñoz, C.A., Dutle, A., Narkawicz, A., Upchurch, J.: Unmanned aircraft systems in the national airspace system: a formal methods perspective. *ACM SIGLOG News* **3**(3), 67–76 (2016)
37. Narkawicz, A., noz, C.M., Dutle, A.: Formally-verified decision procedures for univariate polynomial computation based on sturm’s and tarski’s theorems. *Journal of Automated Reasoning* **54**(4), 285–326 (2015). <https://doi.org/10.1007/s10817-015-9320-x>, pVS formalization of Sturm’s and Tarski’s Theorems; includes sound, complete decision procedures for univariate polynomial inequalities
38. NASA Formal Methods Team, L., the PVS Community: NASALib: NASA PVS Library of Formal Developments. GitHub repository (2025), <https://github.com/nasa/pvslib>, version v7.1.1 (stable release) / current development pre-release

39. Necula, G.C., Lee, P.: The design and implementation of a certifying compiler. In: *Proceedings of the ACM SIGPLAN 1998 Conference on Programming Language Design and Implementation*. p. 333–344. PLDI '98, Association for Computing Machinery, New York, NY, USA (1998). <https://doi.org/10.1145/277650.277752>, <https://doi.org/10.1145/277650.277752>
40. Nipkow, T., Wenzel, M., Paulson, L.C.: *Isabelle/HOL: a proof assistant for higher-order logic*. Springer (2002)
41. Nötzli, A., Barbosa, H., Niemetz, A., Preiner, M., Reynolds, A., Barrett, C., Tinelli, C.: Reconstructing fine-grained proofs of rewrites using a domain-specific language. In: *CONFERENCE ON FORMAL METHODS IN COMPUTER-AIDED DESIGN-FMCAD 2022*. p. 65 (2022)
42. Owre, S., Rushby, J.M., Shankar, N.: Pvs: A prototype verification system. In: *International Conference on Automated Deduction*. pp. 748–752. Springer (1992)
43. Owre, S., Shankar, N.: A brief overview of pvs. In: Mohamed, O.A., Muñoz, C., Tahar, S. (eds.) *Theorem Proving in Higher Order Logics*. pp. 22–27. Springer Berlin Heidelberg, Berlin, Heidelberg (2008)
44. Platzer, A.: Differential dynamic logic for verifying parametric hybrid systems. In: Olivetti, N. (ed.) *TABLEAUX*. LNCS, vol. 4548, pp. 216–232. Springer (2007). https://doi.org/10.1007/978-3-540-73099-6_17
45. Platzer, A.: A complete uniform substitution calculus for differential dynamic logic. *J. Autom. Reas.* **59**(2), 219–265 (2017). <https://doi.org/10.1007/s10817-016-9385-1>
46. Prajna, S., Jadbabaie, A.: Safety verification of hybrid systems using barrier certificates. In: *International Workshop on Hybrid Systems: Computation and Control*. pp. 477–492. Springer (2004)
47. Saillard, R.: Dedukti: a universal proof checker. In: *Foundation of Mathematics for Computer-Aided Formalization Workshop* (2013)
48. Schurr, H.J., Fleury, M., Desharnais, M.: Reliable reconstruction of fine-grained proofs in a proof assistant. In: *CADE*. vol. 28, pp. 450–467 (2021)
49. Shankar, N., Owre, S., Rushby, J.M., Stringer-Calvert, D.W.: Pvs prover guide. Computer Science Laboratory, SRI International, Menlo Park, CA **1**, 11–12 (2001)
50. Slagel, J.T., Moscato, M., White, L., Muñoz, C.A., Balachandran, S., Dutle, A.: Embedding differential dynamic logic in pvs. *Electronic Proceedings in Theoretical Computer Science* **402**, 43–62 (Apr 2024). <https://doi.org/10.4204/eptcs.402.7>, <http://dx.doi.org/10.4204/EPTCS.402.7>
51. Slagel, J.T., White, L., Dutle, A.: Formal verification of semi-algebraic sets and real analytic functions. In: *Proceedings of the 10th ACM SIGPLAN International Conference on Certified Programs and Proofs*. pp. 278–290 (2021)
52. Titolo, L., Feliú, M.A., Moscato, M., Muñoz, C.A.: An abstract interpretation framework for the round-off error analysis of floating-point programs. In: *International Conference on Verification, Model Checking, and Abstract Interpretation*. pp. 516–537. Springer (2017)
53. Yeh, E., Hitaj, B., Owre, S., Quemener, M., Shankar, N.: Coprover: A recommender system for proof construction. In: *Intelligent Computer Mathematics: 16th International Conference, CICM 2023, Cambridge, UK, , September 5–8, 2023 Proceedings*. p. 237–251. Springer-Verlag, Berlin, Heidelberg (2023). https://doi.org/10.1007/978-3-031-42753-4_16, https://doi.org/10.1007/978-3-031-42753-4_16